Zcim Project

Digital Research

CP/M® 1.3 Interface Guide

version 2025-08-05

Printing

Zcim Project edition: version 2025-08-05

Contents

List of Tables

1. Introduction

This manual describes the CP/M system organization including the structure of memory, as well as system entry points. The intention here is to provide the necessary information required to write programs which operate under CP/M, and which use the peripheral and disk I/O facilities of the system

1.1. CP/M Organization

CP/M is logically divided into four parts:

BIOS the basic I/O system for serial peripheral control
BDOS the basic disk operating system primitives
CCP the console command processer
TPA the transient program area

The BIOS and BDOS are combined into a single program with a common entry point and referred to as the FDOS. The CCP is a distinct program which uses the FDOS to provide a human-oriented interface to the information which is cataloged on the diskette. The TPA is an area of memory (i.e, the portion which is not used by the FDOS and CCP) where various non-resident operating system commands are executed. User programs also execute in the TPA. The organization of memory in a standard CP/M system is Table 1

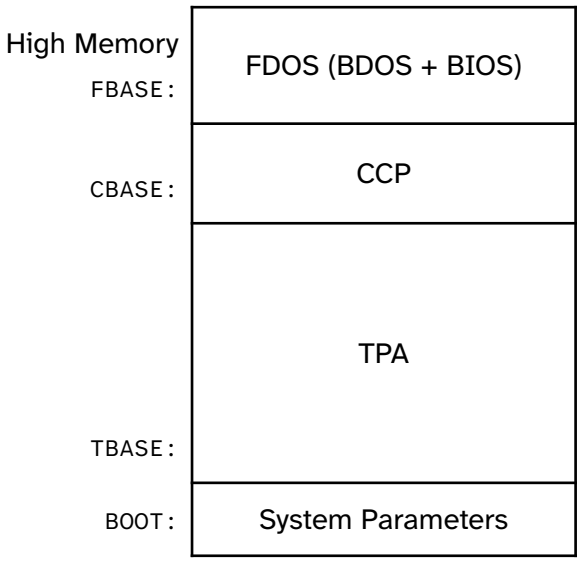

Table 1: CP/M Memory Organization

The lower portion of memory is reserved for system information (which is detailed in later sections), including user defined interrupt locations. The portion between TBASE and CBASE is reserved for the transient operating system commands, while the portion above CBASE contains

the resident CCP and FDOS. The last three locations of memory contain a jump instruction to the FDOS entry point which provides access to system functions.

1.2. Operation of Transient Programs

Transient programs are loaded into the TPA and executed as follows. The operator communicates with the CCP by typing command lines following each prompt. Each command line takes one of the following forms:

command

command file1

command file file2

where *command* is either a built-in function, such as `DIR` or `TYPE`, or the name of a transient command or program. If the *command* is a built-in function of CP/M, it is executed immediately. Otherwise, the CCP searches the currently addressed disk for a file by the name

command`.COM`

If the file is found, it is assumed to be a memory image of a program that executes in the TPA and thus implicitly originates at `TBASE` in memory. The CCP loads the `COM` file from the disk into memory starting at `TBASE` and can extend up to `CBASE`.

If the command is followed by one or two file specifications, the CCP prepares one or two File Control Block (FCB) names in the system parameter area. These optional FCBs are in the form necessary to access files through the FDOS and are described in Section 1.3.

The transient program receives control from the CCP and begins execution, using the I/O facilities of the FDOS. The transient program is called from the CCP. Thus, it can simply return to the CCP upon completion of its processing, or can Jump to `BOOT` to pass control back to CP/M. In the first case, the transient program must not use memory above `CBASE`, while in the latter case, memory up through `FBASE-1` can be used.

The transient program can use the CP/M I/O facilities to communicate with the operator's console and peripheral devices, including the disk subsystem. The I/O system is accessed by passing a function number and an information address to CP/M through the FDOS entry point at `BOOT+0005H`. In the case of a disk read, for example, the transient program sends the number corresponding to a disk read, along with the address of an FCB to the CP/M FDOS. The FDOS, in turn, performs the operation and returns with either a disk read completion indication or an error number indicating that the disk read was unsuccessful.

1.3. Operating System Facilities

CP/M facilities which are available to transients are divided into two categories: BIOS

operations, and BDOS primitives. The BIOS operations are listed first:[1]

- Read Console Character
- Write Console Character
- Read Reader Character
- Write Punch Character
- Write List Device Character
- Set I/O Status
- Interrogate Device Status
- Print Console Buffer
- Read Console Buffer
- Interrogate Console Status

The exact details of BIOS access are given in Section 2. The BDOS primitives include the following operations:

- Disk System Reset
- Drive Select
- File Creation
- File Open
- File Close
- Directory Search
- File Delete
- File Rename
- Read Record
- Write Record
- Interrogate Available Disks
- Interrogate Selected Disk
- Set DMA Address

The details of BDOS access are given in Section 3.

[1]The device support (exclusive of the disk subsystem) corresponds exactly to Intel's peripheral definition, including I/O port assignment and status byte format (see the Intel manual which discusses the Intellec MDS hardware environment).

2. Basic I/O Facilities

Access to common peripherals is accomplished by passing a function number and information address to the BDOS. In general, the function number is passed in Register `C`, while the information address is passed in Register pair `DE`. Note that this conforms to the PL/M Conventions for parameter passing, and thus the following PL/M procedure is sufficient to link to the BIOS when a value is returned:

```
DECLARE ENTRY LITERALLY '0005H'; /* MONITOR ENTRY */
```

```
MON2: PROCEDURE (FUNC, INFO) BYTE;
      DECLARE FUNC BYTE, INFO ADDRESS:
      GO TO ENTRY;
      END MON2;
```

or

```
MON1: PROCEDURE (FUNC, INFO);
      DECLARE FUNC BYTE, INFO ADDRESS:
      GO TO ENTRY;
      END MON1;
```

if no return value is expected.

or

```
MON3: PROCEDURE (FUNC, INFO) ADDRESS;
      DECLARE FUNC BYTE, INFO ADDRESS:
      GO TO ENTRY;
      END MON2;
```

if an address return value is expected.

2.1. Direct and Buffered I/O

The BIOS entry points are given in Table 2. In the case of simple character I/O to the console, the BIOS reads the console device, and removes the parity bit. The character is echoed back to the console, and tab characters (control-I) are expanded to tab positions starting at column one and separated by eight character positions. The I/O status byte takes the form shown in Table I, and can be programmatically interrogated or changed. The buffered read operation takes advantage of the CP/M line editing facilities. That is, the program sends the address of a read buffer whose first byte is the length of the buffer. The second byte is initially empty, but is filled-in by CP/M to the number of characters read from the console after the oper.ation (not including the terminating carriage-return). The remaining positions are used to hold the characters read from the console. The BIOS line editing functions which are performed during this operation are given below:

break line delete and transmit
rubout/DEL delete last character typed, and echo
control-C system reboot
control-U delete entire line
control-E return carriage, but do not transmit buffer (physical carriage return)
<cr> transmit buffer

The read routine also detects control character sequences other than those shown above, and echos them with a preceding ^ symbol. The print entry point allows an entire string of symbols to be printed before returning from the BIOS. The string is terminated by a $ symbol

2.2. A Simple Example

As an example, consider the following PL/M procedures and procedure calls which print a heading, and successively read the console buffer. Each console buffer is then echoed back in reverse order:

```
PRINTCHAR: PROCEDURE (B);
        /* SEND THE ASCII CHARACTER B TO THE CONSOLE */
        DECLARE B BYTE;
        CALL MON1(2,B);
        END PRINTCHAR;

CRLF: PROCEDURE;
        /* SEND CARRIAGE-RETURN-LINE-FEED CHARACTERS */
        CALL PRINTCHAR (0DH); CALL PRINTCHAR (0AH);
        END CRLF;

PRINT: PROCEDURE (A);
        /* PRINT THE BUFFER STARTING AT ADDRESS A */
        DECLARE A ADDRESS;
        CALL MON1(9,A);
        END PRINT;

DECLARE RDBUFF (130) BYTE;

READ: PROCEDURE ;
        /* READ CONSOLE CHARACTERS INTO 'RDBUFF' */
        RDBUFF=128; /* FIRST BYTE SET TO BUFFER LENGTH */
        CALL MON1(l0,.RDBUFF);
        END READ;

DECLARE I BYTE;
CALL CRLF; CALL PRINT (. 'TYPE INPUT LINES $');
        DO WHILE 1; /* INFINITE LOOP-UNTIL CONTROL-C */
        CALL CRLF; CALL PRINTCHAR (' *') /* PROMPT WITH '*' * /
        CALL READ; I = RDBUFF(1);
                DO WHILE (I:= I -1) <"> 255;
                CALL PRINTCHAR (RDBUFF(I+2»;
                END;
        END;
```

The execution of this program might proceed as follows:

```
TYPE INPUT LINES
 *HELLO↵
OLLEH
 *WALL WALLA WASH↵
HSAW ALLAW ALLAW
 *MOM WOW↵
WOW MOM
*^C                                                    (system reboot)
```

#	Name	Parameters	Results
0	System Reset	**none**	**Does not return**
1	Console Input	**none**	**A**: ASCII Character
2	Console Output	**E**: ASCII character to output	**none**
3	Reader Input	**none**	**A**: ASCII Character
4	Punch Output	**E**: ASCII Character	**none**
5	List Output	**E**: ASCII Character	**none**
6	Memory Size	**none**	**HL**: Base Address of the CCP
7	Get IOBYTE	**none**	**A**: IOBYTE value
8	Set IOBYTE	**E**: IOBYTE value	**none**
9	Print String	**DE**: String Address	**none**
10	Read Console Buffer	**DE**: Buffer Address	Characters input are in the Buffer
11	Get Console Status	**none**	**A**: 0FFH if a character is ready, 0 if not
12	Lift Head	**none**	**A**: 00H
13	Reset Disk System	**none**	**A**: 0FFH if $ file present, 0 if not
14	Select Disk	**E**: Drive number: 0 for A, 1 for B, ... 15 for P	**A**: 0 if successful, 0FFH on error
15	Open File	**DE**: FCB Address	**A**: Directory Code
16	Close File	**DE**: FCB Address	**A**: Directory Code
17	Search for First	**DE**: FCB Address	**A**: Directory Code
18	Search for Next	**none**	**A**: Directory Code
19	Delete File	**DE**: FCB Address	**A**: Directory Code
20	Read Sequential	**DE**: FCB Address	**A**: Directory Code
21	Write Sequential	**DE**: FCB Address	**A**: Directory Code
22	Make File	**DE**: FCB Address	**A**: Directory Code
23	Rename File	**DE**: FCB Address	**A**: Directory Code
24	Return Login Vector	**none**	**HL**: Log-in Vector
25	Return Current Disk	**none**	**A**: Current Disk. 0 for A, ... 15 for P
26	Set DMA Address	**DE**: DMA Address	**none**
27	Get Addr(ALLOC)	**none**	**HL**: ALLOC Address

Table 2: CP/M 1.3 Function Summary

3. Disk I/O Facilities

The BDOS section of CP/M provides access to files stored on diskettes. The discussion which follows gives the overall file organization, along with file access mechanisms.

3.1. File System Organization

CP/M implements a named file structure on each diskette, providing a logical organization that allows any particular file to contain any number of records, from completely empty, to the full capacity of a diskette. Each diskette is logically distinct, with complete operating system, disk directory and file data area. The disk file names are in two parts: the *<filename>* (consisting of one to eight alphanumeric characters), and the *<filetype>* (consisting of zero to three alphanumeric characters). The *<filetype>* names the generic category of a particular file, while the filename distinguishes individual files in each category. The *<filetype>*s listed in Table 3 name a few generic categories that have been established, although they are generally arbitrary:

Filetype	Meaning
ASM	Assembler Source
PRN	Printer Listing
HEX	Hex Machine Code
BAS	Basic Source File
INT	Intermediate Code
COM	Command File
PLI	PL/I Source File
REL	Relocatable Module
TEX	TEX Formatter Source
BAK	ED Source Backup
SYM	SID Symbol File
$$$	Temporary File

Table 3: CP/M Filetypes

This, the name

```
X.ASM
```

is interpreted as an assembly language source file by the CCP with *<filename>* X. The files in CP/M are organized as a logically contiguous sequence of 128 byte records (although the records may not be physically contiguous on the diskette), which are normally read or written in sequential order. Random access is allowed under CP/M however, as described in Section 3.4. No particular format within records is assumed by CP/M, although some transients expect particular formats:

1. Source files are treated as a sequence of ASCII characters, where each "line" of the source file is followed by a carriage return, and line-feed sequence (0DH followed by 0AH). Thus, one 128-byte CP/M record can contain several logical lines of source text. Machine code "hex" tapes are also assumed to be in this format, although the loader does not require the carriage return and line-feed characters. The end of an ASCII file is denoted by a CTRL-Z character (1AH) or a real end-of-file returned by the CP/M read operation.
2. COM files are assumed to be absolute machine code in memory image form, starting at TBASE in memory. In this case, control-z is not considered an end of file, but instead is determined by the actual space allocated to the file being accessed.

3.2. File Control Block Format

Each file being accessed through CP/M has a corresponding file control block (FCB) which provides name and allocation information for all file operations. The FCB is a 33-byte area in the transient program's memory space which is set up for each file. The FCB format is given in Figure 2. When accessing CP/M files, it is the programmer's responsibility to fill the lower 16 bytes of the FCB, along with the CR field. Normally, the FN and FT fields are set to the ASCII *<filename>* and *<filetype>*, while all other fields are set to zero. Each FCB describes up to 16K bytes of a particular file (0 to 128 records of 128 bytes each), and, using automatic mechanisms of CP/M, up to 15 additional extensions of the file can be addressed. Thus, each FCB can potentially describe files up to 256K bytes (which is slightly larger than the diskette capacity).

FCB's are stored in a directory area of the diskette, and are brought into central memory before file operations (see the OPEN and MAKE commands) then updated in memory as file operations proceed, and finally recorded on the diskette at the termination of the file operation (see the CLOSE command). This organization makes CP/M file organization highly reliable, since diskette file integrity can only be disrupted in the unlikely case of hardware failure during update of a single directory entry.

It should be noted that the CCP constructs an FCB for all transients by scanning the remainder of the line following the transient name for a *<filename>* or *<filename>.<filetype>* combination.

Any field not specified is assumed to be all blanks. A properly formed FCB is set up at location TFCB (see Section 6), with an assumed I/O buffer at TBUFF. The transient can use TFCB as an address in subsequent input or output operations on this file.

In addition to the default FCB which is set-up at address TFCB, the CCP also constructs a second default FCB at address TFCB+16 (i.e., the disk map field of the FCB at TBASE). Thus, if the user types

```
PROGNAME X.ZOT Y.ZAP
```

the file PROGNAME.COM is loaded to the TPA, and the default FCB at TFCB is initialized to the filename X with filetype ZOT. Since the user typed a second file name, the 16 byte area beginning at TFCB + 16 is also initialized with the filename Y and filetype ZAP. It is the responsibility of the program to move this second filename and filetype to another area (usually a separate file control block) before opening the file which begins at TFCB, since the open operation will fill the disk map portion, thus overwriting the second name and type. If no file names were specified in the original command, then the fields beginning at TFCB and TFCB + 16 both contain blanks (20H). If one file name was specified, then the field at TFCB + 16 contains blanks. If the filetype is omitted, then the field is assumed to contain blanks. In all cases, the CCP translates lower case alphabetic characters to upper case to be consistent with the CP/M file naming conventions. As an added programming convenience, the default buffer at TBUFF is initialized to hold the entire command line past the program name. Address TBUFF contains the number of characters, and TBUFF+1, TBUFF+2, ... , contain the remaining characters up to, but not including, the carriage return. Given that the above command has been typed at the console, the area beginning at TBUFF is set up as follows:

Offset	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	90 ... FF
Content	0EH	*sp*	'B'	':'	'X'	'.'	'Z'	'O'	'T'	*sp*	'Y'	'.'	'Z'	'A'	'P'	00H	???

Table 4: Command-line Layout

where 0EH (12) is the number of valid characters (in binary), and *sp* represents an ASCII blank. Characters are given in ASCII upper case, with uninitialized memory following the last valid character. Again, it is the responsibility of the program to extract the information from this buffer before any file operations are performed since the FDOS uses the TBUFF area to perform directory functions.

In a standard CP/M system, the following values are assumed:

Id	Address	Description
BOOT	0000H	bootstrap load (warm start)
ENTRY	0005H	entry point to FDOS
TFCB	OO5CH	first default file control block

Id	Address	Description
TFCB+16	006CH	second file name
TBUFF	0080H	default buffer address
TBASE	0100H	base of transient area

Field	ET	F1 … F8	T1	T2	T3	EX	S1	S2	RC	D0 … D15	NR
Decimal	00	01 … 08	09	10	11	12	13	14	15	16 … 31	32
Hex	00	01 … 08	09	0A	0B	0C	0D	0E	0F	10 … 1F	20

Table 5: File Control Block Format

Field	Definition
ET	Entry type (currently not used, but assumed to be zero)
F1 … F8	File name, padded with ASCII blanks
T1 .. T3	File type, padded with ASCII blanks
EX	File extent number, normally set to zero
S1	Not used, but assumed zero
S2	Not used, but assumed zero
RC	record count for extent **EX**; takes on values from `0 … 128`
D0 … D15	Disk allocation map, filled in and used by CP/M
NR	Next record to read or write

Table 6: File Control Block Fields

3.3. Disk Access Primitives

Given that a program has properly initialized the FCB's for each of its files, there are several operations which can be performed, as shown in Table 2. In each case, the operation is applied to the currently selected disk (see the disk select operation in Section 5), using the file information in a specific FCB. The following PL/M program segment, for example, copies the contents of the file "`X.Y`" to.the (new) file "`NEW.FIL`":

```
DECLARE RET BYTE;

OPEN:   PROCEDURE (A)
        DECLARE A ADDRESS;
```

```
        RET=MON2(15, A);
        END OPEN;

CLOSE:  PROCEDURE (A);
        DECLARE A ADDRESS;
        RET=MON2(16, A);
        END;

MAKE:   PROCEDURE (A);
        DECLARE A ADDRESS;
        RET=MON2(22, A);
        END MAKE;

DELETE: PROCEDURE (A);
        DECLARE A ADDRESS;
        /* IGNORE RETURNED VALUE */
        CALL MON1(19, A);
        END DELETE;

READBF: PROCEDURE (A);
        DECLARE A ADDRESS;
        RET=MON2 (20, A) ;
        END READBF;

WRITEBF: PROCEDURE (A);
        DECLARE A ADDRESSi
        RET=MON2 (21, A) ;
        END WRITEBF;

INIT:   PROCEDURE;
        CALL MON1(13 , 0);
        END INIT;

/* SET UP FILE CONTROL BLOCKS */
DECLARE FCB1 (33) BYTE
        INITIAL (0, 'X       ', 'Y  ', 0, 0, 0, 0);
        FCB2 (33) BYTE
        INITIAL (0, 'NEW     ', 'FIL', 0 ,0, 0, 0);

CALL INIT;
/* ERASE 'NEW.FIL' IF IT EXISTS */
CALL DELETE (.FCB2);
/* CREATE' 'NEW.FIL' AND CHECK SUCCESS */
CALL MAKE (.FCB2) ;
IF RET = 255 THEN CALL PRINT (.' NO DIRECTORY SPACE $');
      ELSE
      DO;   /* FILE SUCCESSFULLY CREATED, NOW OPEN 'X.Y' */
      CALL OPEN (.FCB1);
      IF RET = 255 THEN CALL PRINT (. 'FILE NOT PRESENT $');
          ELSE
          DO; /* FILE X.Y FOUND AND OPENED, SET
          NEXT RECORD TO ZERO FOR BOTH FILES */
          FCB1(32), FCB2(32) = 0;
          /* READ FILE X.Y UNTIL EOF OR ERROR */
          CALL READBF (.FCB1); /*READ TO 80H*/
              DO WHILE RET = 0;
              CALL WRITEBF (.FCB2) /*WRITE FROM 80H*/
              IF RET = 0 THEN /*GET ANOTHER RECORD*/
```

```
                    CALL READBF (.FCB1); ELSE
                    CALL PRINT (. 'DISK WRITE ERROR $');
                END;
            IF RET <>1 THEN CALL PRINT (.' TRANSFER ERROR $');
            ELSE
                DO; CALL CLOSE (. FCB2) ;
                IF RET = 255 THEN CALL PRINT (.'CLOSE ERROR$');
                END;
            END;
    END;
EOF
```

This program consists of a number of utility procedures for opening, closing, creating, and deleting files, as well as two procedures for reading and writing data. These utility procedures are followed by two FCB's for the input and output files. In both cases, the first 16 bytes are initialized to the *<filename>* and *<filetype>* of the input and output files. The main program first initializes the disk system, then deletes any existing copy of "`NEW.FIL`" before starting. The next step is to create a new directory entry (and empty file) for "`NEW.FIL`". If file creation is successful, the input file "`X.Y`" is opened. If this second operation is also successful, then the disk to disk copy can proceed. The **NR** fields are set to zero so that the first record of each file is accessed on subsequent disk I/O operations. The first call to `READBF` fills the (implied) DMA buffer at `80H` with the first record from "`X.Y`". The loop which follows copies the record at `80H` to "`NEW.FIL`" and then reports any errors, or reads another 128 bytes from "`X.Y`". This transfer operation continues until either all data has been transferred, or an error condition arises. If an error occurs, it is reported; otherwise the new file is closed and the program halts.

3.4. Random Access

Recall that a single FCB describes up to a 16K segment of a (possibly) larger file. Random access within the first 16K segment is accomplished by setting the **NR** field to the record number of the record to be accessed before the disk I/O takes place. Note, however, that if the 128th record is written, then the BOOS automatically increments the extent field (**EX**), and opens the next extent, if possible. In this case, the program must explicitly decrement the **EX** field and re-open the previous extent. If random access outside the first 16K segment is necessary, then the extent number e be explicitly computed, given an absolute record number r as

$$e = \frac{r}{128}$$

or equivalently,

$$e = \mathrm{SHR}(r, 7)$$

this extent number is then placed in the **EX** field before the segment is opened. The **NR** value n is then computed as

$$n = r \bmod 128$$

or

$$n = r \text{ AND } 7\mathrm{F}_{16}$$

When the programmer expects considerable cross-segment accesses, it may save time to create an FCB for each of the 16K segments, open all segments for access, and compute the relevant FCB from the absolute record number r

4. System Generation

As mentioned previously, every diskette used under CP/M is assumed to contain the entire system (excluding transient commands) on the first two tracks. The operating system need not be present, however, if the diskette is only used as secondary disk storage on drives B, C, ... , since the CP/M system is loaded only from drive A.

The CP/M file system is organized so that an IBM-compatible diskette from the factory (or from a vendor which claims IBM compatibility) looks like a diskette with an empty directory. Thus, the user must first copy a version of the CP/M system from an existing diskette to the first two tracks of the new diskette, followed by a sequence of copy operations, using PIP, which transfer the transient command files from the original diskette to the new diskette.

NOTE: before you begin the CP/M copy operation, read your Licensing Agreement. It gives your exact legal obligations when making reproductions of CP/M in whole or in part, and specifically requires that you place the copyright notice

```
Copyright (c), 1976
  Digital Research
```

on each diskette which results from the copy operation.

4.1. Initializing CP/M from an Existing Diskette

The first two tracks are placed on a new diskette by running the transient command SYSGEN, as described in the document "*An Introduction to CP/M Features and Facilities*." The SYSGEN operation brings the CP/M system from an initialized diskette into memory, and then takes the memory image and places it on the new diskette.

Upon completion of the SYSGEN operation, place the original diskette on drive A, and the initialized diskette on drive B. Reboot the system; the response should be:

```
A>
```

indicating that drive A is active. Log into drive B by typing B: and CP/M should respond with:

```
B>
```

indicating that drive B is active. If the diskette in drive B is factory fresh, it will contain an empty directory. Non-standard diskettes may, however, appear as full directories to CP/M, which can be emptied by typing

```
ERA *.*
```

when the diskette to be initialized is active. Do not give the ERA command if you wish to preserve files on the new diskette since all files will be erased with this command.

After examining disk B, reboot the CP/M system and return to drive A for further operations.

The transient commands are then copied from drive A to drive B using the PIP program. The sequence of commands shown below, for example, copy the principal programs from a standard CP/M diskette to the new diskette:

```
A>PIP↵
*B:STAT.COM=STAT.COM↵
*B:PIP.COM=PIP.COM↵
*B:LOAD.COM=LOAD.COM↵
*B:ED.COM=ED.COM↵
*B:ASM.COM=ASM.COM↵
*B:SYSGEN.COM=SYSGEN.COM↵
*B:DDT.COM=DDT.COM↵
*↵
```

The user should then log in disk B, and type the command:

```
DIR *.*
```

to ensure that the files were transferred to drive B from drive A. The various programs can then be tested on drive B to check that they were transferred properly.

Note that the copy operation can be simplified somewhat by creating a “submit” file which contains the copy commands. The file could be named GEN.SUB, for example, and might contain:

```
SYSGEN↵
PIP B:STAT.COM=STAT.COM↵
PIP B:PIP.COM=PIP.COM↵
PIP B:LOAD.COM=LOAD.COM↵
PIP B:ED.COM=ED.COM↵
PIP B:ASM.COM=ASM.COM↵
PIP B:SYSGEN.COM=SYSGEN.COM↵
PIP B:DDT.COM=DDT.COM↵
↵
```

The generation of a new diskette from the standard diskette is then done by typing simply

```
SUBMIT GEN
```

5. CP/M Entry Point Summary

5.1. System Reset

Function 0: System Reset

Entry Parameters:	
Register C:	0 = 00H
Returned Value:	**Does not return**
Typical PL/M Call:	`CALL MON1(0, 0)`

The **System Reset** function returns control to the CP/M operating system at the CCP level. The CCP re-initializes the disk subsystem by selecting and logging-in disk drive A. This function has exactly the same effect as a jump to location 0000H (BOOT).

5.2. Console Input

Function 1: Console Input

Entry Parameters:	
Register C:	1 = 01H
Returned Value:	
Register A:	ASCII Character
Typical PL/M Call:	`I = MON2(1,O)`

The **Console Input** function reads the next console character to register A. Graphic characters, along with carriage return, line-feed, and back space (CTRL-H) are echoed to the console. Tab characters, CTRL-I, move the cursor to the next tab stop. A check is made for start/stop scroll, CTRL-S, and start/stop printer echo, CTRL-P. The FDOS does not return to the calling program until a character has been typed, thus suspending execution if a character is not ready.

5.3. Console Output

Function 2: Console Output

Entry Parameters:	
Register C:	2 = 02H
Register E:	ASCII character to output
Returned Value:	**none**
Typical PL/M Call:	`CALL MON1(2, 'A')`

The **Console Output** function sends the ASCII character from register E to the console device. As in Function 1, tabs are expanded and checks are made for start/stop scroll and printer echo.

5.4. Reader Input

Function 3: Reader Input

Entry Parameters:	
Register C:	3 = 03H
Returned Value:	
Register A:	ASCII Character
Typical PL/M Call:	`I = MON2(3, 0)`

The **Reader Input** function reads the next character from the logical reader into register A. Control does not return until the character has been read.

5.5. Punch Output

Function 4: Punch Output

Entry Parameters:	
Register C:	4 = 04H
Register E:	ASCII Character
Returned Value:	**none**
Typical PL/M Call:	`CALL MON1(4, 'B')`

The **Punch Output** function sends the character from register E to the logical punch device.

5.6. List Output

Function 5: List Output

Entry Parameters:	
Register C:	5 = 05H
Register E:	ASCII Character
Returned Value:	**none**
Typical PL/M Call:	`CALL MON1(5, 'C')`

The **List Output** function sends the ASCII character in register E to the logical listing device.

5.7. Memory Size

Function 6: Memory Size

Entry Parameters:	
Register C:	6 = 06H
Returned Value:	
Registers HL:	Base Address of the CCP
Typical PL/M Call:	`A = MON3(6, 0)`

The **Memory Size** function returns the base address of the Console Command Processor (CCP) in HL.

5.8. Get IOBYTE

Function 7: Get IOBYTE

Entry Parameters:	
Register C:	7 = 07H
Returned Value:	
Register A:	IOBYTE value
Typical PL/M Call:	`IOSTAT = MON2(7, 0)`

The **Get IOBYTE** function returns the current value of IOBYTE in register A.

5.9. Set IOBYTE

Function 8: Set IOBYTE

Entry Parameters:	
Register C:	8 = 08H
Register E:	IOBYTE value
Returned Value:	**none**
Typical PL/M Call:	`CALL MON1(8, IOSTAT)`

The **Set IOBYTE** function changes the IOBYTE value to that given in register E.

5.10. Print String

Function 9: Print String

Entry Parameters:	
Register C:	9 = 09H
Registers DE:	String Address
Returned Value:	**none**
Typical PL/M Call:	`CALL MON1(9, .'PRINT THIS$')`

The **Print String** function sends the character string stored in memory at the location given by DE to the console device, until a '$' (24H) is encountered in the string. Tabs are expanded as in Function 2, and checks are made for start/stop scroll and printer echo.

5.11. Read Console Buffer

Function 10: Read Console Buffer

Entry Parameters:	
Register C:	10 = 0AH
Registers DE:	Buffer Address
Returned Value:	Characters input are in the Buffer
Typical PL/M Call:	`CALL MON1(10, .RDBUFF)`

The **Read Buffer** functions reads a line of edited console input into a buffer addressed by registers DE. Console input is terminated when either input buffer overflows or a carriage return or line-feed is typed. The Read Buffer takes the form:

DE:	+0	+1	+2	+3	+4	+5	+6	+7	+8	...	+n
	mx	**nc**	*c1*	*c2*	*c3*	*c4*	*c5*	*c6*	*c7*	...	??

where **m** is the maximum number of characters that the buffer will hold, 1 to 255, and **nc** is the number of characters read (set by FDOS upon return) followed by the characters read from the console. If $\mathbf{nc} < \mathbf{mx}$, then uninitialized positions follow the last character, denoted by ?? in the above figure. A number of control functions, summarized in Table 7, are recognized during line editing.

Table 7: Edit Control Characters

Character	**Meaning**
CTRL-C	reboots when at beginning of line
CTRL-E	causes physical end of line
CTRL-H	backspaces one character position
CTRL-J	(line-feed) terminates input line
CTRL-M	(carriage return) terminates input line
CTRL-R	retypes the current line after new line
CTRL-U	removes current line
CTRL-X	Same as CTRL-U.
rubout/del	Deletes and echoes the last character typed at the console.

The user should also note that certain functions that return the carriage to the leftmost position (for example, CTRL-X) do so only to the column position where the prompt ended. In earlier releases, the carriage returned to the extreme left margin. This convention makes operator data input and line correction more legible.

5.12. Get Console Status

Function 11: Get Console Status

Entry Parameters:	
Register C:	11 = 0BH
Returned Value:	
Register A:	0FFH if a character is ready, 0 if not
Typical PL/M Call:	I = MON2(11, 0)

The **Console Status** function checks to see if a character has been typed at the console. If a character is ready, the value 0FFH is returned in register A. Otherwise a 00H value is returned.

5.13. Lift Head

Function 12: Lift Head

Entry Parameters:	
Register C:	12 = 0CH
Returned Value:	
Register A:	00H
Registers HL:	FCBDSK variable address or 0
Typical PL/M Call:	CALL MON2(12, 0)

5.14. Reset Disk System

Function 13: Reset Disk System

Entry Parameters:	
Register C:	13 = 0DH
Returned Value:	
Register A:	0FFH if $ file present, 0 if not
Typical PL/M Call:	CALL MON1(13, 0)

The **Reset Disk** function is used to programmatically restore the file system to a reset state where all disks are set to Read-Write. See functions 28 and 29, only disk drive A is selected, and the default DMA address is reset to BOOT+0080H. This function can be used, for example, by an application program that requires a disk change without a system reboot.

This function is normally called by the CCP after the system reset and before any other activity. If the CCP is not executed, this function must be called by what replaced it.

In 1.x and 2.x version the return value depends upon the presence of a file with a name starting with a dollar-sign ($) on drive A. If such a file is present, a 0FFH value is returned, otherwise a zero is returned.

5.15. Select Disk

Function 14: Select Disk

Entry Parameters:	
Register C:	14 = 0EH
Register E:	Drive number: 0 for A, 1 for B, ... 15 for P
Returned Value:	
Register A:	0 if successful, 0FFH on error
Typical PL/M Call:	`CALL MON1(14, 1)`

The **Select Disk** function designates the disk drive named in register E as the default disk for subsequent file operations, with E = 0 for drive A, 1 for drive B, and so on through 15, corresponding to drive P in a full 16 drive system. The drive is placed in an on-line status, which activates its directory until the next cold start, warm start, or disk system reset operation. If the disk medium is changed while it is on-line, the drive automatically goes to a Read-Only status in a standard CP/M environment, see Function 28. FCBs that specify drive code zero (**DR** = 00H) automatically reference the currently selected default drive. Drive code values between 1 and 16 ignore the selected default drive and directly reference drives A through P.

5.16. Open File

Function 15: Open File

Entry Parameters:	
Register C:	15 = 0FH
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	`I = MON2(15, .FCB)`

The **Open** File operation is used to activate a file that currently exists in the disk directory for the currently active user number. The FDOS scans the referenced disk directory for a match in positions 1 through 14 of the FCB referenced by DE (byte **S1** is automatically zeroed) where an ASCII question mark '?' (3FH) matches any directory character in any of these positions. Normally, no question marks are included, and bytes **EX** and **S2** of the FCB are zero.

If a directory element is matched, the relevant directory information is copied into bytes **D0** through **Dn** of FCB, thus allowing access to the files through subsequent read and write operations. The user should note that an existing file must not be accessed until a successful

open operation is completed. Upon return, the open function returns a directory code with the value 0 through 3 if the open was successful or 0FFH (255 decimal) if the file cannot be found. If question marks occur in the FCB, the first matching FCB is activated. Note that the current record, (**CR**) must be zeroed by the program if the file is to be accessed sequentially from the first record.

5.17. Close File

Function 16: Close File

Entry Parameters:	
Register C:	16 = 10H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	I = MON2(16, .FCB)

The **Close** File function performs the inverse of the **Open** File function. Given that the FCB addressed by DE has been previously activated through an **Open** or **Make** function, the **Close** function permanently records the new FCB in the reference disk directory see functions 15 and 22. The FCB matching process for the close is identical to the open function. The directory code returned for a successful close operation is 0, 1, 2, or 3, while a 0FFH (255 decimal) is returned if the filename cannot be found in the directory. A file need not be closed if only read operations have taken place. If write operations have occurred, the close operation is necessary to record the new directory information permanently.

5.18. Search for First

Function 17: Search for First

Entry Parameters:	
Register C:	17 = 11H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	I = MON2(17, .FCB)

Search for First scans the directory for a match with the file given by the FCB addressed by DE. The value 255 (hexadecimal 0FFH) is returned if the file is not found; otherwise, 0, 1, 2, or 3 is returned indicating the file is present. When the file is found, the current DMA address is filled with the record containing the directory entry, and the relative starting position is $A * 32$ (that is, rotate the A register left 5 bits, or ADD A five times). Although not normally required for application programs, the directory information can be extracted from the buffer at this position.

An ASCII question mark '?' (63 decimal, 3F hexadecimal) in any position from **F1** through **EX** matches the corresponding field of any directory entry on the default or auto-selected disk drive. If the **DR** field contains an ASCII question mark, the auto disk select function is disabled and the default disk is searched, with the search function returning any matched entry, allocated or free, belonging to any user number. This latter function is not normally used by application programs, but it allows complete flexibility to scan all current directory values. If the **DR** field is not a question mark, the **S2** byte is automatically zeroed.

5.19. Search for Next

Function 18: Search for Next

Entry Parameters:	
Register C:	18 = 12H
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	`I = MON2(18, .FCB)`

The **Search for Next** function is similar to the **Search for First** function, except that the directory scan continues from the last matched entry. Similar to Function 17, Function 18 returns the decimal value 255 in A when no more directory items match.

5.20. Delete File

Function 19: Delete File

Entry Parameters:	
Register C:	19 = 13H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	`I = MON2(19, .FCB)`

The **Delete** File function removes files that match the FCB addressed by DE. The filename and type may contain ambiguous references (that is, question marks in various positions), but the drive select code (**DR**) cannot be ambiguous, as in the **Search for First** and **Search for Next** functions.

Function 19 returns a decimal 255 if the referenced file or files cannot be found; otherwise, a value in the range 0 to 3 returned.

5.21. Read Sequential

Function 20: Read Sequential

Entry Parameters:	
Register C:	`20 = 14H`
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	`I = MON2(20, .FCB)`

Given that the FCB addressed by DE has been activated through an **Open** or **Make** function, the **Read Sequential** function reads the next 128-byte record from the file into memory at the current DMA address. The record is read from position **CR** of the extent, and the **CR** field is automatically incremented to the next record position. If the **CR** field overflows, the next logical extent is automatically opened and the **CR** field is reset to zero in preparation for the next read operation. The value 00H is returned in the A register if the read operation was successful, while a nonzero value is returned if no data exist at the next record position (for example, end-of-file occurs).

5.22. Write Sequential

Function 21: Write Sequential

Entry Parameters:	
Register C:	`21 = 15H`
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	`I = MON2(21, .FCB)`

Given that the FCB addressed by DE has been activated through an **Open** or **Make** function, the **Write Sequential** function writes the 128-byte data record at the current DMA address to the file named by the FCB. The record is placed at position **CR** of the file, and the **CR** field is automatically incremented to the next record position. If the **CR** field overflows, the next logical extent is automatically opened and the **CR** field is reset to zero in preparation for the next write operation. Write operations can take place into an existing file, in which case, newly written records overlay those that already exist in the file. Register A = 00H upon return from a successful write operation, while a nonzero value indicates an unsuccessful write caused by a full disk.

5.23. Make File

Function 22: Make File

Entry Parameters:	
Register C:	22 = 16H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	`I = MON2(22, .FCB)`

The **Make** File operation is similar to the **Open** File operation except that the FCB must name a file that does not exist in the currently referenced disk directory (that is, the one named explicitly by a nonzero **DR** code or the default disk if **DR** is zero). The FDOS creates the file and initializes both the directory and main memory value to an empty file. The programmer must ensure that no duplicate filenames occur, and a preceding delete operation is sufficient if there is any possibility of duplication. Upon return, register A = 0, 1, 2, or 3 if the operation was successful and 0FFH (255 decimal) if no more directory space is available. The Make function has the side effect of activating the FCB and thus a subsequent open is not necessary.

5.24. Rename File

Function 23: Rename File

Entry Parameters:	
Register C:	23 = 17H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Typical PL/M Call:	`I = MON2(23, .FCB)`

The **Rename** function uses the FCB addressed by DE to change all occurrences of the file named in the first 16 bytes to the file named in the second 16 bytes. The drive code **DR** at position 0 is used to select the drive, while the drive code for the new filename at position 16 of the FCB is assumed to be zero. Upon return, register A is set to a value between 0 and 3 if the rename was successful and 0FFH (255 decimal) if the first filename could not be found in the directory scan.

5.25. Return Login Vector

Function 24: Return Login Vector

Entry Parameters:	
Register C:	24 = 18H
Returned Value:	
Registers HL:	Log-in Vector
Typical PL/M Call:	I = MON2(24, 0)

The log-in vector value returned by CP/M is a 16-bit value in HL, where the least significant bit of L corresponds to the first drive A and the high-order bit of H corresponds to the sixteenth drive, labeled P. A 0 bit indicates that the drive is not on-line, while a 1 bit marks a drive that is actively on-line as a result of an explicit disk drive selection or an implicit drive select caused by a file operation that specified a nonzero **DR** field. The user should note that compatibility is maintained with earlier releases, because registers A and L contain the same values upon return.

5.26. Return Current Disk

Function 25: Return Current Disk

Entry Parameters:	
Register C:	25 = 19H
Returned Value:	
Register A:	Current Disk. 0 for A, ... 15 for P
Typical PL/M Call:	I = MON2(25, 0))

Function 25 returns the currently selected default disk number in register A. The disk numbers range from 0 through 15 corresponding to drives A through P.

5.27. Set DMA Address

Function 26: Set DMA Address

Entry Parameters:	
Register C:	26 = 1AH
Registers DE:	DMA Address
Returned Value:	**none**
Typical PL/M Call:	CALL MON1(26, 2000H)

DMA is an acronym for Direct Memory Address, which is often used in connection with disk controllers that directly access the memory of the mainframe computer to transfer data to and from the disk subsystem. Although many computer systems use non-DMA access (that is, the data is transferred through programmed I/O operations), the DMA address has, in CP/M, come to mean the address at which the 128-byte data record resides before a disk write and after a disk read. Upon cold start, warm start, or disk system reset, the DMA address is automatically

set to BOOT+0080H. The **Set DMA** function can be used to change this default value to address another area of memory where the data records reside. Thus, the DMA address becomes the value specified by DE until it is changed by a subsequent **Set DMA** function, cold start, warm start, or disk system reset.

5.28. Get Addr(ALLOC)

Function 27: Get Addr(ALLOC)

Entry Parameters:	
Register C:	27 = 1BH
Returned Value:	
Registers HL:	ALLOC Address
Typical PL/M Call:	A = MON3(27, 0)

An allocation vector (ALLOC) is maintained in main memory for each on-line disk drive. Various system programs use the information provided by the allocation vector to determine the amount of remaining storage (see the STAT program). Function 27 returns the base address of the allocation vector for the currently selected disk drive. However, the allocation information might be invalid if the selected disk has been marked Read-Only. This function is not normally used by application programs.

6. Address Assignments

The standard distribution version of CP/M is organized for an Intel MDS microcomputer developmental system with 16K of main memory, and two diskette drives. Larger systems are available in 16K increments, providing management of 32K, 48K, and 64K systems (the largest MDS system is 62K since the ROM monitor provided with the MDS resides in the top 2K of the memory space). For each additional 16K increment, add 4000H to the values of CBASE and FBASE.

Id	Address	Description
BOOT	0000H	bootstrap load (warm start)
TFCB	OO5CH	first default file control block
TBUFF	0080H	default buffer address
TBASE	0100H	base of transient area
CBASE	2900H	base of console command processor
FBASE	3200H	base of disk operating system
ENTRY	0005H	entry point to disk system from user programs

7. Sample Programs

This section contains two sample programs which interface with the CP/M operating system. The first program is written in assembly language, and is the source program for the DUMP utility. The second program is the CP/M LOAD utility, written in PL/M.

The assembly language program begins with a number of “equates” for system entry points and program constants. The equate

```
BDOS EQU 0005H
```

for example, gives the CP/M entry point for peripheral I/O functions. The default file control block address is also defined (FCB) , along with the default buffer address (BUFF). Note that the program is set up to run at location 100H, which is the base of the transient program area. The stack is first set-up by saving the entry stack pointer into OLDSP, and resetting SP to the local stack. The stack pointer upon entry belongs to the console command processor, and need not be saved unless control is to return to the CCP upon exit. That is, if the program terminates with a reboot (branch to location 0000H) then the entry stack pointer need not be saved.

The program then jumps to MAIN, past a number of subroutines which are listed below:

BREAK when called, checks to see if there is a console character ready. BREAK is used to stop the listing at the console.

PCHAR print the character which is in register A at the console.

CRLF send carriage return and line feed to the console.

PNIB print the hexadecimal value in register A in ASCII at the console.

PHEX print the byte value (two ASCII characters) in register A at the console.

ERR - print error flag #n at the console, where n is

- **1** if file cannot be opened
- **2** if disk read error occurred

GNB get next byte of data from the input file. If the IBP (input buffer pointer) exceeds the size of the input buffer, then another disk record of 128 bytes is read. Otherwise, the next character in the buffer is returned. IBP is updated to point to the next character.

The MAIN program then appears, which begins by calling SETUP. The SETUP subroutine, discussed below, opens the input file and checks for errors. If the file is opened properly, the GLOOP (get loop) label gets control. On each successive pass through the GLOOP label, the next data byte is fetched using GNB and saved in register B. The line addresses are listed every sixteen bytes, so there must be a check to see if the least significant 4 bits is zero on each output. If so, the line address is taken from registers H and L, and typed at the left of the line. In all cases, the byte which was previously saved in register B is brought back to register A, following label NONUM, and

printed in the output line. The cycle through GLOOP continues until an end of file condition is detected in DISKR, as described below. Thus, the output lines appear as:

```
0000 bb bb bb bb bb bb bb bb bb bb bb bb bb bb bb bb
0010 bb bb bb bb bb bb bb bb bb bb bb bb bb bb bb bb
```

until the end of file.

The label FINIS gets control upon end of file. CRLF is called first to return the carriage from the last line output. The CCP-stack pointer is then reclaimed from OLDSP, followed by a RET to return to the console command processor. Note that a JMP 0000H could be used following the FINIS label, which would cause the CP/M system to be brought in again from the diskette (this operation is necessary only if the CCP has been over- layed by data areas).

The file control block format is then listed (FCBDN FCBLN) which overlays the FCB at location 05CH which is setup by the CCP when the DUMP program is initiated. That is, if the user types

```
DUMP X.Y
```

then the CCP sets up a properly formed FCB at location 05CH for the DUMP (or any other) program when it goes into execution. Thus, the SETUP subroutine simply addresses this default FCB, and calls the disk system to open it. The DISKR (disk read) routine is called whenever GNB needs another buffer full of data. The default buffer at location 80H is used, along with a pointer (IBP) which counts bytes as they are processed. Normally, an end of file condition is taken as either an ASCII 1AH (control-Z), or an end of file detection by the OS. The file dump program, however, stops only on a CP/M end of file.

```
                ;       FILE DUMP PROGRAM, READS AN INPUT FILE AND PRINTS IN HEX
                ;
                ;       COPYRIGHT (C), DIGITAL RESEARCH, 1975, 1976
                ;
0100                    ORG     100H
0005 =          BDOS    EQU     0005H   ;DOS ENTRY POINT
000F =          OPENF   EQU     15      ;FILE OPEN
0014 =          READF   EQU     20      ;READ FUNCTION
0002 =          TYPEF   EQU     2       ;TYPE FUNCTION
0001 =          CONS    EQU     1       ;READ CONSOLE
000B =          BRKF    EQU     11      ;BREAK KEY FUNCTION (TRUE IF CHAR READY)
                ;
005C =          FCB     EQU     5CH     ;FILE CONTROL BLOCK ADDRESS
0080 =          BUFF    EQU     80H     ;INPUT DISK BUFFER ADDRESS
                ;
                ;       SET UP STACK
0100 210000             LXI     H,0
0103 39                 DAD     SP
0104 220F01             SHLD    OLDSP
0107 315101             LXI     SP,STKTOP
010A C3C401             JMP     MAIN
                ;       VARIABLES
010D            IBP:    DS      2       ;INPUT BUFFER POINTER
                ;
                ;       STACK AREA
010F            OLDSP:  DS      2
0111            STACK:  DS      64
0151 =          STKTOP  EQU     $
                ;
                ;       SUBROUTINES
                ;
                BREAK:  ;CHECK BREAK KEY (ACTUALLY ANY KEY WILL DO)
0151 E5D5C5             PUSH H! PUSH D! PUSH B; ENVIRONMENT SAVED
0154 0E0B               MVI     C,BRKF
0156 CD0500             CALL    BDOS
0159 C1D1E1             POP B! POP D! POP H; ENVIRONMENT RESTORED
015C C9                 RET
                ;
                PCHAR:  ;PRINT A CHARACTER
015D E5D5C5             PUSH H! PUSH D! PUSH B; SAVED
0160 0E02               MVI     C,TYPEF
0162 5F                 MOV     E,A
0163 CD0500             CALL    BDOS
0166 C1D1E1             POP B! POP D! POP H; RESTORED
0169 C9                 RET
                ;
                CRLF:
016A 3E0D               MVI     A,0DH
016C CD5D01             CALL    PCHAR
016F 3E0A               MVI     A,0AH
0171 CD5D01             CALL    PCHAR
0174 C9                 RET
                ;
                ;
                PNIB:   ;PRINT NIBBLE IN REG A
0175 E60F               ANI     0FH     ;LOW 4 BITS
0177 FE0A               CPI     10
0179 D28101             JNC     P10
                ;       LESS THAN OR EQUAL TO 9
017C C630               ADI     '0'
017E C38301             JMP     PRN
                ;
                ;       GREATER OR EQUAL TO 10
0181 C637       P10:    ADI     'A' - 10
0183 CD5D01     PRN:    CALL    PCHAR
0186 C9                 RET
                ;
                PHEX:   ;PRINT HEX CHAR IN REG A
0187 F5                 PUSH    PSW
0188 0F                 RRC
0189 0F                 RRC
018A 0F                 RRC
018B 0F                 RRC
018C CD7501             CALL    PNIB    ;PRINT NIBBLE
018F F1                 POP     PSW
0190 CD7501             CALL    PNIB
0193 C9                 RET
```

```
                ;
                ERR:    ;PRINT ERROR MESSAGE
0194 CD6A01             CALL    CRLF
0197 3E23               MVI     A, '#'
0199 CD5D01             CALL    PCHAR
019C 78                 MOV     A, B
019D C630               ADI     '0'
019F CD5D01             CALL    PCHAR
01A2 CD6A01             CALL    CRLF
01A5 C3F701             JMP     FINIS
                ;
                GNB:    ;GET NEXT BYTE
01A8 3A0D01             LDA     IBP
01AB FE80               CPI     80H
01AD C2B401             JNZ     G0
                ;       READ ANOTHER BUFFER
                ;
                ;
01B0 CD1602             CALL    DISKR
01B3 AF                 XRA     A
                ;
                G0:     ;READ THE BYTE AT BUFF+REG A
01B4 5F                 MOV     E,A
01B5 1600               MVI     D,0
01B7 3C                 INR     A
01B8 320D01             STA     IBP
                ;       POINTER IS INCREMENTED
                ;       SAVE THE CURRENT FILE ADDRESS
01BB E5                 PUSH    H
01BC 218000             LXI     H,BUFF
01BF 19                 DAD     D
01C0 7E                 MOV     A,M
                ;       BYTE IS IN THE ACCUMULATOR
                ;
                ;       RESTORE FILE ADDRESS AND INCREMENT
01C1 E1                 POP     H
01C2 23                 INX     H
01C3 C9                 RET
                ;
                MAIN:   ; READ AND PRINT SUCCESSIVE BUFFERS
01C4 CDFF01             CALL    SETUP   ; SET UP INPUT FILE
01C7 3E80               MVI     A,80H
01C9 320D01             STA     IBP     ; SET BUFFER POINTER TO 80H
01CC 21FFFF             LXI     H,0FFFFH        ;SET TO -1 TO START
                ;
                GLOOP:
01CF CDA801             CALL    GNB
01D2 47                 MOV     B,A
                ;       PRINT HEX VALUES

                ;       CHECK FOR LINE FOLD
01D3 7D                 MOV     A,L
01D4 E60F               ANI     0FH     ;CHECK LOW 4 BITS
01D6 C2EB01             JNZ     NONUM
                ;       PRINT LINE NUMBER
01D9 CD6A01             CALL    CRLF
                ;
                ;       CHECK FOR BREAK KEY
01DC CD5101             CALL    BREAK
01DF 0F                 RRC
01E0 DAF701             JC      FINIS   ;DON'T PRINT ANY MORE
                ;
01E3 7C                 MOV     A,H
01E4 CD8701             CALL    PHEX
01E7 7D                 MOV     A,L
01E8 CD8701             CALL    PHEX
                NONUM:
01EB 3E20               MVI     A,' '
01ED CD5D01             CALL    PCHAR
01F0 78                 MOV     A,B
01F1 CD8701             CALL    PHEX

01F4 C3CF01             JMP     GLOOP
```

```
                ;
                EPSA:   ;END PSA
                        ;               END OF INPUT
                FINIS:
01F7 CD6A01             CALL    CRLF
01FA 2A0F01             LHLD    OLDSP
01FD F9                 SPHL
01FE C9                 RET
                ;
                ;
                ;       FILE CONTROL BLOCK DEFINITIONS
005C =          FCBDN   EQU     FCB+0   ;DISK NAME
005D =          FCBFN   EQU     FCB+1   ;FILE NAME
0065 =          FCBFT   EQU     FCB+9   ;DISK FILE TYPE (3 CHARACTERS)
0068 =          FCBRL   EQU     FCB+12  ;FILE'S CURRENT REEL NUMBER
006B =          FCBRC   EQU     FCB+15  ;FILE'S RECORD COUNT (0 TO 128)
007C =          FCBCR   EQU     FCB+32  ;CURRENT (NEXT) RECORD NUMBER (0 TO 127)
007D =          FCBLN   EQU     FCB+33  ;FCB LENGTH
                ;
                SETUP:  ;SET UP FILE
                ;       OPEN THE FILE FOR INPUT
01FF 115C00             LXI     D,FCB
0202 0E0F               MVI     C,OPENF
0204 CD0500             CALL    BDOS
                ;       CHECK FOR ERRORS
0207 FEFF               CPI     255
0209 C21102             JNZ     OPNOK
                ;       BAD OPEN
020C 0601               MVI     B, 1    ;OPEN ERROR
020E CD9401             CALL    ERR
                ;
                OPNOK:  ;OPEN OPERATION OK, SET BUFFER INDEX TO END
0211 AF                 XRA     A
0212 327C00             STA     FCBCR
0215 C9                 RET
                ;
                DISKR:  ;READ DISK FILE RECORD
0216 E5D5C5             PUSH H! PUSH D! PUSH B
0219 115C00             LXI     D,FCB
021C 0E14               MVI     C,READF
021E CD0500             CALL    BDOS
0221 C1D1E1             POP B! POP D! POP H
0224 FE00               CPI     0       ;CHECK FOR ERRS
0226 C8                 RZ
                ;       MAY BE EOF
0227 FE01               CPI     1
0229 CAF701             JZ      FINIS
                ;
022C 0602               MVI     B,2     ;DISK READ ERROR
022E CD9401             CALL    ERR
                ;
0231                    END
```

The PL/M program which follows implements the CP/M LOAD utility. The function is as follows. The user types

LOAD *filename*

If *filename*.HEX exists on the diskette, then the LOAD utility reads the “hex” formatted machine code file and produces the file

filename.COM

where the COM file contains an absolute memory image of the machine code, ready for load and execution in the TPA. If the file does not appear on the diskette, the LOAD program types

SOURCE IS READER

and reads an Addmaster paper tape reader which contains the hex file. The LOAD program is set up to load and run in the TPA, and, upon completion, return to the CCP without rebooting the system. Thus, the program is constructed as a single procedure called LOADCOM which takes the form:

```
0FAH:
    LOADCOM: PROCEDURE;
        /* LIBRARY PROCEDURES */
        MON1: ...
        /* END LIBRARY PROCEDURES */
        MOVE: ...
        GETCHAR: ...
        PRINTNIB: ...
        PRINTHEX: ...
        PRINTADDR: ...
        RELOC: ...
            SETMEM:
            READHEX:
            READBYTE:
            READCS:
            MAKE$DOUBLE:
            DIAGNOSE:
        END RELOC;

        DECLARE STACK(16) ADDRESS, SP ADDRESS;
        SP = STACKPOINTER; STACKPOINTER = .STACK(LENGTH(STACK));

        ...
        CALL RELOC;
        ...
        STACKPOINTER = SP;
        RETURN 0;
    END LOADCOM;
;
EOF
```

The label 0FAH at the beginning sets the origin of the compilation to 0FAH, which causes the first 6 bytes of the compilation to be ignored when loaded (i.e., the TPA starts at location 100H and thus 0FAH, ... ,0FFH are deleted from the COM file). In a PL/M compilation, these 6 bytes are used to set up the stack pointer and branch around the subroutines in the program. In this case, there is only one subroutine, called LOADCOM, which results in the following machine memory image for LOAD

```
0FAH:   LXI   SP,plmstack     ;SET SP TO DEFAULT STACK
0FDH:   JMP   pastsubr        ;JUMP AROUND LOADCOM
100H:   beginning of LOADCOM procedure
        end of LOADCOM procedure
        RET

pastsubr:
        EI
        HLT
```

Since the machine code between 0FAH and 0FFH is deleted in the load, execution actually begins at the top of LOADCOM. Note, however, that the initialization of the SP to the default stack has also been deleted; thus, there is a declaration and initialization of an explicit stack and stack pointer before the call to RELOC at the end of LOADCOM. This is necessary only if we wish to return to the CCP without a reboot operation: otherwise the origin of the program is set to 100H, the declaration of LOADCOM as a procedure is not necessary, and termination is accomplished by simply executing a

```
GO TO 0000H;
```

at the end of the program. Note also that the overhead for a system re- boot is not great (approximately 2 seconds), but can be bothersome for system utilities which are used quite often, and do not need the extra space.

The procedures listed in LOAOCOM as “library procedures” are a standard set of PL/M subroutines which are useful for CP/M interface. The RELOC procedure contains several nested subroutines for local functions, and actually performs the load operation when called from LOADCOM. Control initially starts on line 327 where the stack pointer is saved and re-initialized to the local stack. The default file control block name is copied to another file control block (SFCB) since two files may be open at the same time. The program then calls SEARCH to see if the HEX file exists; if not, then the high speed reader is used. If the file does exist, it is opened for input (if possible). The filetype COM is moved to the default file control block area, and any existing copies of filename. COM files are removed from the diskette before creating a new file. The MAKE operation creates a new file, and, if successful, RELOC is called to read the HEX file and produce the COM

file. At the end of processing by RELOC, the COM file is closed (line 350). Note that the HEX file does not need to be closed since it was opened for input only. The data written to a file is not permanently recorded until the file is successfully closed.

Disk input characters are read through the procedure GETCHAR on line 137. Although the DMA facilities of CP/M could be used here, the GETCHAR procedure instead uses the default buffer at location 80H and moves each buffer into a vector called SBUFF (source buffer) as it is read. On exit, the GETCHAR procedure returns the next input character and updates the source buffer pointer (SBP).

The SETMEM procedure on line 191 performs the opposite function from GETCHAR. The SETMEM procedure maintains a buffer of loaded machine code in pure binary form which acts as a "window" on the loaded code. If there is an attempt by RELOC to write below this window, then the data is ignored. If the data is within the window, then it is placed into MBUFF (memory buffer). If the data is to be placed above this window, then the window is moved up to the point where it would include the data address by writing the memory image successively (by 128 byte buffers), and moving the base address of the window. Using this technique, the programmer can recover from checksum errors on the high-speed reader by stopping the reader, rewinding the tape for seme distance, then restarting LOAD (in this case, *LOADing* is resumed by interrupting with a NOP instruction). Again, the SETMEM procedure uses the default buffer at location 80H to perform the disk output by moving 128 byte segments to 80H through 0FFH before each write.

```
00001  1   0FAH: DECLARE BDOS LITERALLY '0005H';
00002  1     /* TRANSIENT COMMAND LOADER PROGRAM
00003  1
00004  1                   COPYRIGHT (C) DIGITAL RESEARCH
00005  1                           JUNE, 1975
00006  1     */
00007  1
00008  1
00009  1   LOADCOM: PROCEDURE BYTE;
00010  2       DECLARE FCBA ADDRESS INITIAL(5CH);
00011  2       DECLARE FCB BASED FCBA (33) BYTE;
00012  2
00013  2       DECLARE BUFFA ADDRESS INITIAL(80H), /* I/O BUFFER ADDRESS */
00014  2           BUFFER BASED BUFFA (128) BYTE;
00015  2
00016  2       DECLARE SFCB(33) BYTE,  /* SOURCE FILE CONTROL BLOCK */
00017  2           BSIZE LITERALLY '1024',
00018  2           EOFILE LITERALLY '1AH',
00019  2           SBUFF(BSIZE) BYTE  /* SOURCE FILE BUFFER */
00020  2               INITIAL(EOFILE),
00021  2           RFLAG BYTE,         /* READER FLAG */
00022  2           SBP ADDRESS;        /* SOURCE FILE BUFFER POINTER */
00023  2
00024  2       /* LOADCOM LOADS TRANSIENT COMMAND FILES TO THE DISK FROM THE
00025  2       CURRENTLY DEFINED READER PERIPHERAL.  THE LOADER PLACES THE MACHINE
00026  2       CODE INTO A FILE WHICH APPEARS IN THE LOADCOM COMMAND */
00027  2   /* **************** LIBRARY PROCEDURES FOR DISKIO ************ */
00028  2
00029  2   MON1: PROCEDURE (F,A);
00030  3       DECLARE F BYTE,
00031  3       A ADDRESS;
00032  3       GO TO BDOS;
00033  3       END MON1;
00034  2
00035  2   MON2: PROCEDURE (F,A) BYTE;
00036  3       DECLARE F BYTE,
00037  3       A ADDRESS;
00038  3       GO TO BDOS;
00039  3       END MON2;
00040  2
00041  2   READRDR: PROCEDURE BYTE;
00042  3       /* READ CURRENT READER DEVICE */
00043  3       RETURN MON2(3,0);
00044  3       END READRDR;
00045  2
00046  2   DECLARE
00047  2       TRUE LITERALLY '1',
00048  2       FALSE LITERALLY '0',
00049  2       FOREVER LITERALLY 'WHILE TRUE',
00050  2       CR LITERALLY '13',
00051  2       LF LITERALLY '10',
00052  2       WHAT LITERALLY '63';
00053  2
00054  2   PRINTCHAR: PROCEDURE (CHAR);
00055  3       DECLARE CHAR BYTE;
00056  3       CALL MON1(2,CHAR);
00057  3       END PRINTCHAR;
00058  2
00059  2   CRLF: PROCEDURE;
00060  3       CALL PRINTCHAR(CR);
00061  3       CALL PRINTCHAR(LF);
00062  3       END CRLF;
00063  2
00064  2   PRINT: PROCEDURE (A);
00065  3       DECLARE A ADDRESS;
00066  3       /* PRINT THE STRING STARTING AT ADDRESS A UNTIL THE
00067  3       NEXT DOLLAR SIGN IS ENCOUNTERED */
00068  3       CALL CRLF;
00069  3       CALL MON1 (9,A) ;
00070  3       END PRINT;
00071  2
00072  2   DECLARE DCNT BYTE;
00073  2
00074  2   INITIALIZE: PROCEDURE;
00075  3       CALL MON1 (13,0);
00076  3       END INITIALIZE;
00077  2
```

```
00078  2   SELECT: PROCEDURE (D) ;
00079  3       DECLARE D BYTE;
00080  3       CALL MON1 (14, D) ;
00081  3       END SELECT;
00082  2
00083  2   OPEN: PROCEDURE (FCB);
00084  3       DECLARE FCB ADDRESS;
00085  3       DCNT = MON2(15,FCB);
00086  3       END OPEN;
00087  2
00088  2   CLOSE: PROCEDURE (FCB);
00089  3       DECLARE FCB ADDRESS;
00090  3       DCNT = MON2(16,FCB);
00091  3       END CLOSE;
00092  2
00093  2   SEARCH: PROCEDURE (FCB);
00094  3       DECLARE FCB ADDRESS;
00095  3       DCNT = MON2(17,FCB);
00096  3       END SEARCH;
00097  2
00098  2   SEARCHN: PROCEDURE;
00099  3       DCNT = MON2(18,0);
00100  3       END SEARCHN;
00101  2
00102  2   DELETE: PROCEDURE (FCB) ;
00103  3       DECLARE FCB ADDRESS;
00104  3       CALL MON1(19,FCB);
00105  3       END DELETE;
00106  2
00107  2   DISKREAD: PROCEDURE (FCB) BYTE;
00108  3       DECLARE FCB ADDRESS;
00109  3       RETURN MON2(20,FCB);
00110  3       END DISKREAD;
00111  2
00112  2   DISKWRITE: PROCEDURE (FCB) BYTE;
00113  3       DECLARE FCB ADDRESS;
00114  3       RETURN MON2(21,FCB);
00115  3       END DISKWRITE;
00116  2
00117  2   MAKE: PROCEDURE (FCB) ;
00118  3       DECLARE FCB ADDRESS;
00119  3       DCNT = MON2(22,FCB);
00120  3       END MAKE;
00121  2
00122  2   RENAME: PROCEDURE (FCB) ;
00123  3       DECLARE FCB ADDRESS;
00124  3       CALL MON1(23,FCB);
00125  3       END RENAME;
00126  2
00127  2   /* ****************** END OF LIBRARY PROCEDURES ************* */
00128  2
00129  2       MOVE: PROCEDURE(S,D,N);
00130  3           DECLARE (S,D) ADDRESS, N BYTE,
00131  3           A BASED S BYTE, B BASED D BYTE;
00132  3               DO WHILE (N:=N-1) <> 255;
00133  3               B = A; S=S+1; D=D+1;
00134  4               END;
00135  3           END MOVE;
00136  2
00137  2       GETCHAR: PROCEDURE BYTE;
00138  3           /* GET NEXT CHARACTER */
00139  3           DECLARE I BYTE;
00140  3           IF RFLAG THEN RETURN READRDR;
00141  3           IF (SBP := SBP+1) <= LAST(SBUFF) THEN
00142  3               RETURN SBUFF(SBP);
00143  3           /* OTHERWISE READ ANOTHER BUFFER FULL */
00144  3               DO SBP = 0 TO LAST(SBUFF) BY 128;
00145  3               IF (I:=DISKREAD(.SFCB)) = 0 THEN
00146  4                   CALL MOVE(80H,.SBUFF(SBP),80H); ELSE
00147  4                   DO; IF I<>1 THEN CALL PRINT(.'DISK READ ERROR$');
00148  5                   SBUFF(SBP) = EOFILE;
00149  5                   SBP = LAST(SBUFF);
00150  5                   END;
00151  4               END;
00152  3           SBP = 0; RETURN SBUFF;
00153  3           END GETCHAR;
```

```
00154  2   DECLARE
00155  2       STACKPOINTER LITERALLY 'STACKPTR';
00156  2
00157  2
00158  2   PRINTNIB: PROCEDURE(N);
00159  3       DECLARE N BYTE;
00160  3           IF N > 9 THEN CALL PRINTCHAR(N+'A'-10); ELSE
00161  3           CALL PRINTCHAR(N+'0');
00162  3       END PRINTNIB;
00163  2
00164  2   PRINTHEX: PROCEDURE(B);
00165  3       DECLARE B BYTE;
00166  3           CALL PRINTNIB(SHR(B,4)); CALL PRINTNIB(B AND 0FH);
00167  3       END PRINTHEX;
00168  2
00169  2   PRINTADDR: PROCEDURE(A);
00170  3       DECLARE A ADDRESS;
00171  3       CALL PRINTHEX(HIGH(A)); CALL PRINTHEX(LOW(A));
00172  3       END PRINTADDR;
00173  2
00174  2
00175  2    /* INTEL HEX FORMAT LOADER */
00176  2
00177  2   RELOC: PROCEDURE;
00178  3       DECLARE (RL, CS, RT) BYTE;
00179  3       DECLARE
00180  3           LA ADDRESS,     /* LOAD ADDRESS */
00181  3           TA ADDRESS,     /* TEMP ADDRESS */
00182  3           SA ADDRESS,     /* START ADDRESS */
00183  3           FA ADDRESS,     /* FINAL ADDRESS */
00184  3           NB ADDRESS,     /* NUMBER OF BYTES LOADED */
00185  3           SP ADDRESS,     /* STACK POINTER UPON ENTRY TO RELOC */
00186  3
00187  3           MBUFF(256) BYTE,
00188  3           P BYTE,
00189  3           L ADDRESS;
00190  3
00191  3           SETMEM: PROCEDURE(B);
00192  4               /* SET MBUFF TO B AT LOCATION LA MOD LENGTH(MBUFF) */
00193  4               DECLARE (B,I) BYTE;
00194  4               IF LA < L THEN /* MAY BE A RETRY */ RETURN;
00195  4                   DO WHILE LA > L + LAST(MBUFF); /* WRITE A PARAGRAPH */
00196  4                       DO I = 0 TO 127; /* COPY INTO BUFFER */
00197  5                       BUFFER(I) = MBUFF(LOW(L)); L = L + 1;
00198  6                       END;
00199  5                   /* WRITE BUFFER ONTO DISK */
00200  5                   P = P + 1;
00201  5                   IF DISKWRITE(FCBA) <> 0 THEN
00202  5                       DO; CALL PRINT(.'DISK WRITE ERROR$');
00203  6                       HALT;
00204  6                       /* RETRY AFTER INTERRUPT NOP */
00205  6                       L = L - 128;
00206  6                       END;
00207  5                   END;
00208  4               MBUFF(LOW(LA)) = B;
00209  4               END SETMEM;
00210  3
00211  3           READHEX: PROCEDURE BYTE;
00212  4               /* READ ONE HEX CHARACTER FROM THE INPUT */
00213  4               DECLARE H BYTE;
00214  4               IF (H := GETCHAR) - '0' <= 9 THEN RETURN H - '0';
00215  4               IF H - 'A' > 5 THEN GO TO CHARERR;
00216  4               RETURN H - 'A' + 10;
00217  4               END READHEX;
00218  3
00219  3           READBYTE: PROCEDURE BYTE;
00220  4               /* READ TWO HEX DIGITS */
00221  4               RETURN SHL(READHEX,4) OR READHEX;
00222  4               END READBYTE;
00223  3
00224  3           READCS: PROCEDURE BYTE;
00225  4               /* READ BYTE WHILE COMPUTING CHECKSUM */
00226  4               DECLARE B BYTE;
00227  4               CS = CS + (B := READBYTE);
00228  4               RETURN B;
00229  4               END READCS;
00230  3
```

```
00231  3          MAKE$DOUBLE: PROCEDURE(H,L) ADDRESS;
00232  4              /* CREATE A BOUBLE BYTE VALUE FROM TWO SINGLE BYTES */
00233  4              DECLARE (H,L) BYTE;
00234  4              RETURN SHL(DOUBLE(H),8) OR L;
00235  4              END MAKE$DOUBLE;
00236  3
00237  3          DIAGNOSE: PROCEDURE;
00238  4
00239  4          DECLARE M BASED TA BYTE;
00240  4
00241  4          NEWLINE: PROCEDURE;
00242  5              CALL CRLF; CALL PRINTADDR(TA); CALL PRINTCHAR(':');
00243  5              CALL PRINTCHAR(' ');
00244  5              END NEWLINE;
00245  4
00246  4          /* PRINT DIAGNOSTIC INFORMATION AT THE CONSOLE */
00247  4              CALL PRINT(.'LOAD  ADDRESS $'); CALL PRINTADDR(TA);
00248  4              CALL PRINT(.'ERROR ADDRESS $'); CALL PRINTADDR(LA);
00249  4
00250  4              CALL PRINT(.'BYTES READ:$'); CALL NEWLINE;
00251  4              DO WHILE TA < LA;
00252  4              IF (LOW(TA) AND 0FH) = 0  THEN CALL NEWLINE;
00253  5              CALL PRINTHEX(MBUFF(TA-L)); TA=TA+1;
00254  5              CALL PRINTCHAR(' ');
00255  5              END;
00256  4          CALL CRLF;
00257  4          HALT;
00258  4          END DIAGNOSE;
00259  3
00260  3
00261  3          /* INITIALIZE */
00262  3          SA, FA, NB = 0;
00263  3          SP = STACKPOINTER;
00264  3          P = 0; /* PARAGRAPH COUNT */
00265  3          TA,LA,L = 100H;  /* BASE ADDRESS OF TRANSIENT ROUTINES */
00266  3          IF FALSE THEN
00267  3              CHARERR:  /* ARRIVE HERE IF NON-HEX DIGIT IS ENCOUNTERED */
00268  3              DO; /* RESTORE STACKPOINTER */ STACKPOINTER = SP;
00269  4              CALL PRINT(.'NON-HEXADECIMAL DIGIT ENCOUNTERED $');
00270  4              CALL DIAGNOSE;
00271  4              END;
00272  3
00273  3
00274  3          /* READ RECORDS UNTIL :00XXXX IS ENCOUNTERED */
00275  3
00276  3              DO FOREVER;
00277  3              /* SCAN THE : */
00278  3                  DO WHILE GETCHAR <> ':';
00279  4                  END;
00280  4
00281  4              /* SET CHECK SUM TO ZERO, AND SAVE THE RECORD LENGTH */
00282  4              CS = 0;
00283  4              /* MAY BE THE END OF TAPE */
00284  4              IF (RL := READCS) = 0 THEN
00285  4                     GO TO FIN;
00286  4              NB = NB + RL;
00287  4
00288  4              TA, LA = MAKE$DOUBLE(READCS,READCS);
00289  4              IF SA = 0 THEN SA = LA;
00290  4
00291  4
00292  4              /* READ THE RECORD TYPE (NOT CURRENTLY USED) */
00293  4              RT = READCS;
00294  4
00295  4              /* PROCESS EACH BYTE */
00296  4                  DO WHILE (RL := RL - 1) <> 255;
00297  4                  CALL SETMEM(READCS); LA = LA+1;
00298  5                  END;
00299  4              IF LA > FA THEN FA = LA - 1;
00300  4
00301  4              /* NOW READ CHECKSUM AND COMPARE */
00302  4              IF CS + READBYTE <> 0 THEN
00303  4                  DO; CALL PRINT(.'CHECK SUM ERROR $');
00304  5                  CALL DIAGNOSE;
00305  5                  END;
00306  4              END;
00307  3
```

```
00308  3           FIN:
00309  3           /* EMPTY THE BUFFERS */
00310  3           TA = LA;
00311  3               DO WHILE L < TA;
00312  3               CALL SETMEM(0); LA = LA+1;
00313  4               END;
00314  3           /* PRINT FINAL STATISTICS */
00315  3           CALL PRINT(.'FIRST ADDRESS $'); CALL PRINTADDR(SA);
00316  3           CALL PRINT(.'LAST  ADDRESS $'); CALL PRINTADDR(FA);
00317  3           CALL PRINT(.'BYTES READ    $'); CALL PRINTADDR(NB);
00318  3           CALL PRINT(.'RECORDS WRITTEN $'); CALL PRINTHEX(P);
00319  3           CALL CRLF;
00320  3
00321  3           END RELOC;
00322  2
00323  2       /* ARRIVE HERE FROM THE SYSTEM MONITOR, READY TO READ THE HEX TAPE */
00324  2
00325  2       /* SET UP STACKPOINTER IN THE LOCAL AREA */
00326  2   DECLARE STACK(16) ADDRESS, SP ADDRESS;
00327  2   SP = STACKPOINTER; STACKPOINTER = .STACK(LENGTH(STACK));
00328  2
00329  2   SBP = LENGTH(SBUFF);
00330  2      /* SET UP THE SOURCE FILE */
00331  2       CALL MOVE(FCBA,.SFCB,33);
00332  2       CALL MOVE(.('HEX',0),.SFCB(9),4);
00333  2       CALL SEARCH(.SFCB);
00334  2       IF (RFLAG := DCNT = 255) THEN
00335  2           CALL PRINT(.'SOURCE IS READER$'); ELSE
00336  2           DO; CALL PRINT(.'SOURCE IS DISK$');
00337  3           CALL OPEN(.SFCB);
00338  3           IF DCNT = 255 THEN CALL PRINT(.'-CANNOT OPEN SOURCE$');
00339  3           END;
00340  2       CALL CRLF;
00341  2
00342  2       CALL MOVE(.'COM',FCBA+9,3);
00343  2
00344  2    /* REMOVE ANY EXISTING FILE BY THIS NAME */
00345  2    CALL DELETE(FCBA);
00346  2    /* THEN OPEN A NEW FILE */
00347  2    CALL MAKE(FCBA); CALL OPEN(FCBA);
00348  2    IF DCNT = 255 THEN CALL PRINT(.'NO MORE DIRECTORY SPACE$'); ELSE
00349  2           DO; CALL RELOC;
00350  3           CALL CLOSE(FCBA);
00351  3           IF DCNT = 255 THEN CALL PRINT(.'CANNOT CLOSE FILE$');
00352  3           END;
00353  2    CALL CRLF;
00354  2
00355  2    /* RESTORE STACKPOINTER FOR RETURN */
00356  2   STACKPOINTER = SP;
00357  2   RETURN 0;
00358  2   END LOADCOM;
00359  1   ;
00360  1   EOF
```

Notes on the Zcim Project Edition

The Zcim Project goal is to improve the accessibility of legacy Z80 CP/M era content. As part of the project, Jim Burlingame (*jb@samplx.org*) created a *Typst* version of this manual.

You can find out more about the project at its site z80cim.org. The sources of this document are available on GitHub.

The source material is from the *Tim Olmstead Memorial Digital Research CP/M Library*

The individual documents from the archive include:

- *CP/M 1.4 Interface Guide* (PDF)

Additional documents are available from z80pack, including:

- *CP/M 1.3 Interface Guide* (PDF)

The contents of the manual were edited using the Typst.app site.

The sources of the document are available at GitHub.

License